

Bulacan State University

College of Science

MCS 306 & TWM 301: Introduction to Artificial Intelligence & Technical Writing in Mathematics

Academic Year 2026–2027, First Semester



MINI RESEARCH AND EXPOSITION PAPER

Skill-Based Gig Matching for GigLink Using TF-IDF, Latent Semantic Analysis, and Optimal Assignment

Authors

Kristiana T. Evangelista, Thealyne G. Jacinto, Jammele E. Perez, Tifany B. Santos, and Justine S. Segubiense

BSM CS 3B – G2

Submitted to:

Aarhus M. Dela Cruz

October 31, 2026

Abstract

GigLink is a proposed web-based marketplace in which students and young professionals of Bulacan State University take on short, task-based gigs posted by clients. One of its planned modules is *provider matching*: given a gig posting, recommend the seekers who are most likely to have the required skills. This paper formulates provider matching as a content-based ranking problem followed by a one-to-one assignment problem. Gig descriptions and seeker profiles are represented as TF-IDF vectors in a vector space model; latent semantic analysis (LSA), computed with a randomized truncated singular value decomposition, maps them into a lower-dimensional concept space. A hybrid score combines LSA cosine similarity with the Jaccard similarity of skill tags, and when many gigs are open at the same time the linear assignment problem is solved exactly with a Hungarian-type algorithm. Because GigLink has not yet been deployed, the method is evaluated on real public data instead of synthetic data: 42,029 Upwork job posts serve as gigs and 2,483 LiveCareer resumes serve as seeker profiles, with relevance judged by a hand-written table that links each Upwork job category to the resume categories that do that work. Over 20 random samples of 400 gigs, the tuned hybrid model reached a test nDCG@10 of 0.438 (95% CI $\pm$0.011) and a precision at 5 of 0.389, compared with 0.406 for LSA alone, 0.362 for TF-IDF, 0.324 for tag matching and 0.068 for random ranking; it beat every baseline on all 20 samples. Accuracy differed strongly between kinds of work, from about 0.75 for design and marketing gigs to below 0.2 for data-analytics and support gigs. Exact assignment collected 0.9% (many seekers) to 6.1% (few seekers) more relevance than greedy assignment, never double-booked a seeker, and ran faster. The study shows that simple, explainable linear-algebra methods work on real job and resume texts and are a feasible first AI component for GigLink, while also showing clearly where they are weak.

Keywords: recommender systems, vector space model, TF-IDF, latent semantic analysis, singular value decomposition, linear assignment problem, gig economy, job matching

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem statement . . . 1
1.3 Objectives and research questions . . . 2
1.4 Why this model . . . 2
1.5 Scope and data statement . . . 2

2 Preliminaries **2**
2.1 Vector space model and TF-IDF . . . 3
2.2 Singular value decomposition and latent semantic analysis . . . 3
2.3 Set similarity . . . 4
2.4 The linear assignment problem . . . 4
2.5 Ranking evaluation metrics . . . 4

3 Methods and Applications **5**
3.1 Model description . . . 5
3.2 Dataset . . . 6
3.3 Experimental procedure . . . 7
3.4 Implementation and code . . . 7

4 Results and Visualization **10**
4.1 Ranking quality (RQ1) . . . 10
4.2 Robustness to shorter profiles and missing tags (RQ2) . . . 12
4.3 Simultaneous assignment (RQ3) . . . 13
4.4 Numerical behaviour and cost (RQ4) . . . 13

5 Analysis and Discussion **15**
5.1 Limitations and sources of error . . . 17

6 Conclusion **18**

7 Recommendations and Future Directions **18**

References **19**

A Reproducing the Results **22**

B Ranking Metrics Code **22**

List of Figures

1 Test-set ranking quality of each method on real data (mean and 95% CI over 20 random samples). 10
2 Validation nDCG@10 as a function of the text weight α (sample 0). 11
3 Mean test nDCG@10 for the 15 most frequent Upwork categories (number of test gigs over all samples in brackets). 12
4 nDCG@10 when resumes are shortened (left) and when seeker tags are deleted at random (right); mean over 5 samples per level. 13
5 Numerical behaviour of latent semantic analysis. 14
6 Statistical convergence of the estimates and computational cost. 15

List of Tables

1 Numerical settings used in all experiments. 8
2 Test-set ranking quality on real Upwork job posts and real resumes (mean $\pm$ half-width of the 95% confidence interval over 20 random samples; 200 test gigs $\times$ 2,483 resumes per sample). Best value per column in bold. 10
3 Paired comparisons over the same 20 random samples (Hybrid minus baseline). 11
4 Ablation of the hybrid model (test gigs, 20 samples, mean $\pm$ 95% CI). α is the weight on text similarity; $1 - \alpha$ goes to tag overlap. 12
5 Simultaneous assignment of all 400 sampled gigs (mean over 20 samples). *True relevance* sums the category-based relevance of the chosen pairs; *good match rate* is the share of the gigs that receive a resume from the primary matching category ($\mathrm{rel} = 2$). 13
6 Randomized SVD versus exact ARPACK singular values ($k = 40$, matrix 2883$\times$41587, mean of 5 random starts). 14
7 Wall-clock time in seconds (median of 3 runs) for n gigs and n seekers. The last row is the slope of a least-squares line fitted to $\log t$ versus $\log n$ for $n \geq 400$ (empirical complexity exponent). 15
8 Status of each component of this study. 17

1. Introduction

1.1. Background

Many university students work while studying, both to cover living and school costs and to gain experience. In the Philippine setting, students commonly look for short-term work through referrals, social-media groups and messaging apps, which makes finding the right worker slow and unreliable for clients. GigLink is a proposed platform where clients post short, task-based gigs (for example tutoring, graphic design, writing, programming, photography, social media management, online selling and administrative support) and student seekers apply for them. A central need of such a platform is an intelligent matching mechanism that helps clients identify suitable seekers based on the skills and requirements specified in a gig posting. This study develops and evaluates that AI-based matching component.

Studies of online labour platforms show that clients rely heavily on profile information such as skills, ratings and rates when they choose freelancers (Ren et al., 2023), and recruitment recommender systems are an active research area (Freire & de Castro, 2021; de Ruijt & Bhulai, 2021). Industrial systems use learned embeddings to match jobs and candidates at large scale (Zhao et al., 2021). A small student platform such as GigLink, however, starts without click logs or hiring histories, so methods that need large amounts of interaction data are not yet usable. Matching people to tasks is a standard application of AI techniques for representing and retrieving information (Russell & Norvig, 2021; Manning et al., 2008). Content-based recommendation, which compares the *content* of items and user profiles (Lops et al., 2011), does not need such history and is therefore a natural starting point.

1.2. Problem statement

When a client posts a gig, GigLink must rank hundreds of seeker profiles. Two practical difficulties make this harder than exact keyword search:

1. **Vocabulary mismatch.** A client may write “pubmat for our org event” while a seeker writes “poster and layout design”; both describe the same skill.
2. **Different people describe themselves differently.** Real profiles vary widely in length, wording and how many skills they list, and clients describe the same kind of work in different ways. A matching method must work with this variety instead of assuming every user behaves the same.

In addition, when many gigs are posted at once the platform should not send the same top seeker to every client, and in the full platform a seeker who is unavailable or too expensive for a gig should not be recommended.

1.3. Objectives and research questions

The general objective is to design, implement and evaluate a mathematically grounded AI matching method for GigLink. Specifically, we aim to answer:

RQ1 How accurately do text-based (TF-IDF, LSA), tag-based (Jaccard) and hybrid models rank seekers, compared with random and keyword baselines?

RQ2 How robust are these models when seeker profiles are shorter or list fewer skills?

RQ3 Does solving a global assignment problem give better simultaneous matches than greedy or independent recommendations?

RQ4 What numerical settings (SVD rank and iteration count) and computational costs does the method require?

1.4. Why this model

We compared four candidate AI features for GigLink: a chatbot assistant, intelligent search, collaborative filtering and content-based skill matching. Collaborative filtering needs interaction history that does not exist yet, and a chatbot is difficult to evaluate quantitatively within the time available. Content-based matching works from the first day, rests on well-understood linear algebra (vectors, inner products, the singular value decomposition) and combinatorial optimisation (the assignment problem), is explainable to users ("matched because of: logo design, poster design"), and directly implements a planned GigLink module. It therefore offers the best balance of feasibility, mathematical depth and research value for this project.

1.5. Scope and data statement

GigLink is not yet deployed, so it has no user data of its own. Instead of generating artificial data, we evaluate the method on two **real, publicly available datasets** (Section 3.2): job posts that clients wrote on the freelancing platform Upwork, which play the role of GigLink gigs, and resumes that real job seekers wrote on the LiveCareer website, which play the role of GigLink seeker profiles. **No synthetic data are used anywhere in this study.** Because no public dataset records who was actually hired for which post, relevance is judged with an explicit category-compatibility table (Section 3.2). The results therefore measure how well the methods match real job texts to real resumes of the right field; they are *not* measurements of GigLink users. Table 8 at the end of Section 5 lists which parts are implemented and tested and which are only proposed.

2. Preliminaries

Throughout, vectors are columns, $\|\boldsymbol{x}\|_2$ is the Euclidean norm and $|A|$ is the size of a finite set A. There are m gigs indexed by g and n seekers indexed by s.

2.1. Vector space model and TF-IDF

In the vector space model (Salton et al., 1975), each document d (a gig description or a seeker bio) becomes a vector in $\mathbb{R}^{|V|}$, where V is the vocabulary of distinct words after removing common English stop-words.

Definition 2.1 (TF-IDF weight)**.** Let $\mathrm{tf}_{t,d}$ be the number of times term t occurs in document d, N the number of documents and df_t the number of documents containing t. With sublinear term frequency and smoothed inverse document frequency (Salton & Buckley, 1988; Manning et al., 2008),

$$w_{t,d} = \big(1 + \ln \mathrm{tf}_{t,d}\big) \cdot \Big(1 + \ln \frac{1+N}{1+\mathrm{df}_t}\Big) \quad (\mathrm{tf}_{t,d} > 0), \qquad w_{t,d} = 0 \text{ otherwise.} \tag{2.1}$$

The document vector is normalised, $\boldsymbol{x}_d = \boldsymbol{w}_d / \|\boldsymbol{w}_d\|_2$.

The idf factor gives rare, informative words ("calculus") more weight than words that appear everywhere ("student").

Definition 2.2 (Cosine similarity)**.** For non-zero $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^p$, $\cos(\boldsymbol{u}, \boldsymbol{v}) = \dfrac{\boldsymbol{u}^\top \boldsymbol{v}}{\|\boldsymbol{u}\|_2 \|\boldsymbol{v}\|_2}$. For unit vectors this is simply the inner product $\boldsymbol{u}^\top \boldsymbol{v}$, and for non-negative TF-IDF vectors it lies in $[0, 1]$.

2.2. Singular value decomposition and latent semantic analysis

Stack the N unit document vectors as rows of $X \in \mathbb{R}^{N \times |V|}$.

Theorem 2.1 (SVD and best low-rank approximation)**.** *Every real matrix has a factorisation $X = U \Sigma V^\top$ with U, V orthogonal and Σ diagonal with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. For $k < \mathrm{rank}(X)$, the truncation $X_k = U_k \Sigma_k V_k^\top$ that keeps the k largest singular values minimises $\|X - Y\|_F$ over all matrices Y of rank at most k, with $\|X - X_k\|_F^2 = \sum_{i>k} \sigma_i^2$ (Eckart & Young, 1936).*

Latent semantic analysis (Deerwester et al., 1990) represents document d by the k-dimensional vector

$$\boldsymbol{z}_d = V_k^\top \boldsymbol{x}_d \in \mathbb{R}^k \quad \text{(equivalently the } d\text{-th row of } U_k \Sigma_k), \qquad \hat{\boldsymbol{z}}_d = \boldsymbol{z}_d / \|\boldsymbol{z}_d\|_2. \tag{2.2}$$

Words that tend to appear in similar documents load on the same singular vectors, so two documents can be close in the latent space even when they share few exact words. This is the property we hope will reduce vocabulary mismatch.

Randomized truncated SVD. Computing only the top k singular triplets is done with the randomized range finder of Halko et al. (2011): draw a Gaussian test matrix $\Omega \in \mathbb{R}^{|V| \times (k+p)}$ (oversampling p), form $Y = (XX^\top)^q X\Omega$ with q *power iterations*, orthonormalise $Y = QR$,

and take the exact SVD of the small matrix $B = Q^\top X$. Power iterations sharpen the decay of the spectrum: in the error bound of Halko et al. (2011), the constant multiplying σ_{k+1} is raised to the power $1/(2q+1)$, so the error approaches the optimal value σ_{k+1} as q grows. The number of power iterations q is the main iteration parameter of our method, and Section 4.4 measures its effect.

2.3. Set similarity

Definition 2.3 (Jaccard similarity)**.** For finite sets A, B, $J(A,B) = |A \cap B|/|A \cup B|$, with $J(\emptyset,\emptyset) = 0$. If $\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^{|\mathcal{K}|}$ are the indicator vectors of A, B over a skill list $\mathcal{K}$, then $|A \cap B| = \boldsymbol{a}^\top \boldsymbol{b}$ and $|A \cup B| = \boldsymbol{a}^\top \mathbf{1} + \boldsymbol{b}^\top \mathbf{1} - \boldsymbol{a}^\top \boldsymbol{b}$, which lets all gig–seeker pairs be computed with one matrix product.

2.4. The linear assignment problem

Given a utility matrix $H \in \mathbb{R}^{m \times n}$, the (rectangular) linear assignment problem is

$$\begin{aligned} &\max_{x} \sum_{g=1}^{m} \sum_{s=1}^{n} H_{gs}\, x_{gs} \\ &\text{s.t. } \textstyle\sum_s x_{gs} \le 1 \;\; \forall g, \qquad \sum_g x_{gs} \le 1 \;\; \forall s, \qquad x_{gs} \in \{0,1\}. \end{aligned} \tag{2.3}$$

Its constraint matrix is totally unimodular, so the linear-programming relaxation already has an integral optimum, and the problem can be solved exactly in polynomial time by the Hungarian method (Kuhn, 1955). SciPy's `linear_sum_assignment` implements a shortest augmenting path variant described by Crouse (2016), with worst-case time $O(n^3)$ for square problems.

2.5. Ranking evaluation metrics

For gig g, let $\pi_g(i)$ be the seeker placed at rank i and $\mathrm{rel}_{g,s} \in \{0,1,2\}$ the graded ground-truth relevance; a seeker is *relevant* when $\mathrm{rel} = 2$. Let $\mathcal{R}_g$ be the set of relevant seekers for g. Then

$$\mathrm{P@}k = \frac{1}{k} \sum_{i=1}^{k} \mathbf{1}[\mathrm{rel}_{g,\pi_g(i)} \ge 2], \qquad \mathrm{R@}k = \frac{1}{|\mathcal{R}_g|} \sum_{i=1}^{k} \mathbf{1}[\mathrm{rel}_{g,\pi_g(i)} \ge 2], \tag{2.4}$$

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{2^{\mathrm{rel}_{g,\pi_g(i)}} - 1}{\log_2(i+1)}, \qquad \mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \tag{2.5}$$

$$\mathrm{RR} = 1/\min\{i : \mathrm{rel}_{g,\pi_g(i)} \ge 2\}, \tag{2.6}$$

where IDCG@k is the DCG of the ideal ordering (Järvelin & Kekäläinen, 2002). Each metric is averaged over gigs (MRR is the mean of RR). All four lie in $[0,1]$ and larger is better.

nDCG@10 is our primary metric because it rewards putting the *most* suitable seekers first and handles graded relevance.

3. Methods and Applications

3.1. Model description

Inputs. For each gig g: the free-text title and description written by the client, and the set of required skill tags T_g that the client selected on Upwork (for example *logo design*, *adobe illustrator*). For each seeker s: the free text of the resume, and a set of skill tags $\tilde{T}_s$. Resumes do not come with tags, so $\tilde{T}_s$ is the set of Upwork skill phrases (from the shared skill list $\mathcal{K}$, Section 3.2) that occur word-for-word in the resume.

Similarity components. Text similarity $c_{gs} = \hat{\boldsymbol{z}}_g^\top \hat{\boldsymbol{z}}_s$ (LSA, Eq. 2.2) or $c_{gs} = \boldsymbol{x}_g^\top \boldsymbol{x}_s$ (TF-IDF, Eq. 2.1), and tag similarity $j_{gs} = J(T_g, \tilde{T}_s)$.

Hybrid score. With a weight $\alpha \in [0, 1]$,

$$H_{gs} = \alpha\, c_{gs} + (1 - \alpha)\, j_{gs}. \tag{3.1}$$

This is a convex combination of two similarities in $[0, 1]$: $\alpha = 1$ uses text only and $\alpha = 0$ uses tags only. For each gig the seekers are ranked by decreasing H_{gs}.

Feasibility constraints (platform design). In the GigLink application a pair should only be recommended when the seeker's schedule overlaps the gig's working days and the expected cost fits the budget,

$$F_{gs} = \mathbf{1}\big[D_g \cap D_s \neq \emptyset\big] \cdot \mathbf{1}\big[\rho_s h_g \leq b_g\big], \tag{3.2}$$

with infeasible pairs moved to the bottom of the ranking ($H_{gs} \leftarrow -\infty$). The code supports this filter, but the public resumes contain neither availability nor hourly rates, so **the constraint could not be evaluated on real data** and every pair is treated as feasible in the experiments ($F \equiv 1$).

Assignment. When all m gigs are matched at once, we solve Problem (2.3) with H from Eq. (3.1). Infeasible pairs, when known, are forbidden by giving them utility -10^6 (any forbidden pair the solver still returns, which only happens when a gig has no feasible seeker left, is discarded).

Assumptions.

Assumption 1. Job posts and resumes mention the skills involved, possibly in different words; there is no deliberate deception.

Assumption 2. A resume is suitable for a gig when the person works in the field that the gig belongs to, as recorded by the category-compatibility table; work quality, price and availability are *not* part of the ground truth (see Section 5.1).

Assumption 3. In the assignment experiment each gig needs one seeker and each seeker takes at most one gig in the matching round.

3.2. Dataset

Why these datasets. Our instructor required real rather than synthetic data, and GigLink itself has no users yet. We therefore chose two public Kaggle datasets that mirror the two sides of the GigLink marketplace. Both were written by real people for real job searches, both are in English, and their records are reproduced unchanged in our pipeline; only parsing and lower-casing are applied.

Gigs: Upwork job posts (asaniczka, 2024). 52,977 job posts collected from Upwork's public job feed between December 2023 and February 2024. Each post has a title, a description, the pay type (hourly or fixed price) with its rate or budget, the client's country, and, embedded in the description text, the Upwork *category* chosen by the client (220 different categories, e.g. *Graphic Design*, *Data Entry*, *Video Editing*) and the Upwork *skill tags* the client attached (median 5 tags per post). Upwork is the largest general freelancing platform, and its short, task-based posts are the closest public analogue to GigLink gigs.

Seekers: LiveCareer resumes (Bhawal, 2021). 2,483 resumes of real job seekers collected from the LiveCareer website, each filed under one of 24 job categories (e.g. DESIGNER, ACCOUNTANT, INFORMATION-TECHNOLOGY, CHEF). Names, companies and contact details are replaced by placeholders in the source, so no personal information is processed. A resume has a median of 757 words. The resumes are much longer and more varied than the job posts, and some categories (chef, aviation, agriculture, healthcare) have no matching Upwork work at all, so they act as realistic distractors.

Skill tags for resumes. The shared skill list $\mathcal{K}$ contains the 2,814 Upwork skill tags of at most four words that occur in at least five job posts. A resume receives a tag when the tag phrase occurs in it as a whole sequence of words (`extract_tags` in `real_data.py`); the median resume receives 24 tags. Some of these tags are generic words such as *business* or *marketing*, which is part of the real difficulty of the task.

Ground truth. No public dataset records who was hired for which Upwork post, so we judge relevance with a **category-compatibility table** (`data/category_map.csv`) that we wrote by hand before running any model. For each of the 94 most common Upwork categories it

names the resume category that does that work (*primary*) and up to three categories that do closely related work (*related*); for example *Logo Design* → DESIGNER (primary), ARTS and DIGITAL-MEDIA (related). Graded relevance is

$$\mathrm{rel}_{gs} = \begin{cases} 2 & \text{the resume's category is the primary category for gig } g, \\ 1 & \text{it is a related category,} \\ 0 & \text{otherwise.} \end{cases} \tag{3.3}$$

The table uses only the two category labels, which no model sees, so the models are graded against an outside judgement rather than against their own inputs. Upwork posts in categories with no clear resume counterpart (for example *General Virtual Assistance* or *Creative Writing*) are left out, which keeps 42,029 of the job posts. In a typical sample, 109 resumes are relevant ($\mathrm{rel} = 2$) to each gig, about 4.4% of all resumes.

Sampling. For each random sample R we draw 400 gigs without replacement from the mapped job posts and rank all 2,483 resumes for each of them. Different samples contain different gigs, so the spread over samples shows how much the results depend on which posts happen to be chosen.

3.3. Experimental procedure

1. For each random sample $R = 0, \ldots, 19$, draw 400 gigs and split them at random into 200 *validation* and 200 *test* gigs; the seeker pool is always all resumes.
2. Compute all score matrices (Random, Keyword, Jaccard, TF-IDF, LSA). *Keyword* counts distinct shared words without weighting; *Random* draws uniform scores.
3. Tune the hybrid model by grid search on the validation gigs only: text source $\in \{\text{TF-IDF}, \text{LSA}\}$ and $\alpha \in \{0, 0.1, \ldots, 1\}$ (22 combinations), maximising nDCG@10.
4. Evaluate every method on the test gigs, which were never used for tuning.
5. Run ablations, robustness sweeps (5 samples per level), the assignment experiment (all 400 sampled gigs), an SVD convergence study and a runtime study.
6. Report mean, standard deviation and a t-based 95% confidence interval over samples, and compare the hybrid with each baseline by a paired t-test and a Wilcoxon signed-rank test (Wilcoxon, 1945) on the same samples.

3.4. Implementation and code

The implementation is in Python 3 with NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), scikit-learn (Pedregosa et al., 2011), pandas and Matplotlib (Hunter, 2007). The full code, with comments, unit tests and instructions, is in the project's Git repository (`giglink_ai/`). Table 1 lists every numerical setting.

Table 1: Numerical settings used in all experiments.

Parameter	Value	Where set
Gigs per sample / seekers	400 / all 2,483 resumes	`RealConfig`
Random samples (main / sweeps)	20 / 5	`experiments.py`
Skill list $\mathcal{K}$	tags used by ≥ 5 posts, ≤ 4 words	`skill_vocabulary`
LSA rank k	40	`LSA_K`
Power iterations q (randomized SVD)	7	`SVD_ITERS`
Oversampling p	10 (scikit-learn default)	`TruncatedSVD`
Exact SVD reference tolerance	0 (machine precision, ARPACK)	`svds(tol=0)`
α grid	$\{0, 0.1, \ldots, 1\}$	`W_TEXT_GRID`
Cut-offs	P@5, R@100, nDCG@10	`metrics.py`
Seekers in the scarce assignment market	300	`N_SCARCE`
Forbidden-pair utility	-10^6	`assignment.py`

Listing 1 shows the LSA scoring step, which implements Eqs. (2.1) and (2.2).

Listing 1: LSA similarity (excerpt of `giglink_ai/models.py`).

```
def lsa_scores(gig_text, seeker_text, k=40, n_iter=7, seed=0):
    _, G, S = fit_tfidf(gig_text, seeker_text) # unit-length TF-IDF rows
    X = vstack([G, S]) # N x |V| document-term matrix
    svd = TruncatedSVD(n_components=k, n_iter=n_iter, random_state=seed)
    Z = normalize(svd.fit_transform(X)) # rows of U_k Sigma_k, unit
        length
    Zg, Zs = Z[:G.shape[0]], Z[G.shape[0]:]
    return Zg @ Zs.T # cosine similarity matrix
```

Listing 2 shows how the real data are turned into the model inputs and the ground truth, and Listing 3 implements Eq. (3.1).

Listing 2: Resume tags and graded relevance (excerpt of `giglink_ai/real_data.py`).

```
def extract_tags(texts, vocab):
    n_max = max(len(v.split()) for v in vocab) # longest skill phrase
    return texts.map(lambda t: ";".join(sorted(_ngrams(t, n_max) & vocab)
        ))

def relevance_matrix(gigs, seekers, cmap):
    rel = np.zeros((len(gigs), len(seekers)), dtype=int)
    scat = seekers.category.to_numpy()
    for i, cat in enumerate(gigs.category):
        primary, related = cmap[cat] # from data/category_map.csv
        rel[i, np.isin(scat, list(related))] = 1
        rel[i, scat == primary] = 2
    return rel
```

Listing 3: Hybrid score (excerpt of `giglink_ai/models.py`).

```
def hybrid_scores(text_sim, tag_sim, rating01, w_text=0.6, w_rating
    =0.05):
    # the experiments use w_rating = 0 because the resumes carry no
        ratings
```

```
    return w_text * text_sim + (1.0 - w_text) * tag_sim + w_rating *
        rating01[None, :]
```

Listing 4 shows the exact and greedy assignment policies. Algorithm 1 summarises the full pipeline.

Listing 4: Optimal and greedy assignment (excerpt of `giglink_ai/assignment.py`).

```
def optimal_assignment(scores, feasible):
    H = np.where(feasible, scores, -1e6)
    rows, cols = linear_sum_assignment(H, maximize=True) # exact solver
    return [(g, s) for g, s in zip(rows, cols) if feasible[g, s]]

def greedy_assignment(scores, feasible):
    H = np.where(feasible, scores, -np.inf)
    used_g, used_s, pairs = set(), set(), []
    for flat in np.argsort(-H, axis=None, kind="stable"): # best pair
        first
        g, s = divmod(int(flat), H.shape[1])
        if not np.isfinite(H[g, s]): break
        if g in used_g or s in used_s: continue
        pairs.append((g, s)); used_g.add(g); used_s.add(s)
    return pairs
```

Algorithm 1 GigLink provider matching

Require: gig texts and Upwork tags, resume texts, skill list $\mathcal{K}$; parameters k, q, α

Build the TF-IDF matrix X of all gig and seeker texts ▷ Eq. (2.1)
Compute a rank-k randomized SVD of X with q power iterations; form unit vectors $\hat{\boldsymbol{z}}_d$ ▷ Eq. (2.2)
Tag each resume with the phrases of $\mathcal{K}$ it contains, giving $\tilde{T}_s$
$C \leftarrow [\hat{\boldsymbol{z}}_g^\top \hat{\boldsymbol{z}}_s]$, $J \leftarrow [J(T_g, \tilde{T}_s)]$
$H \leftarrow \alpha C + (1-\alpha) J$ ▷ Eq. (3.1)
for each gig g **do**
    return seekers sorted by $H_{g\cdot}$ (top-K recommendation list)
end for
if matching many gigs at once **then**
    solve Problem (2.3) on H (infeasible pairs forbidden when known)
end if

Reproducibility. Every random choice comes from a seeded NumPy generator. After the two Kaggle files are placed in `data/raw/`, one command, `python -m giglink_ai.experiments`, recomputes all results and figures (about 17 minutes on the machine used), and `python -m giglink_ai.make_tables` rewrites the tables in this paper from the result files. Nine unit tests check the metrics against hand-computed values, the Jaccard matrix, the parsing of Upwork posts, the resume tagging, the relevance grades and the assignment against SciPy's optimum.

4. Results and Visualization

All results below were **obtained by running the code on the real Upwork job posts and LiveCareer resumes**; none are projected or assumed.

4.1. Ranking quality (RQ1)

Table 2 and Figure 1 compare all methods on the test gigs. The tuned hybrid model had the best mean on every metric: nDCG@10 $= 0.438$ and P@5 $= 0.389$, meaning that on average about 2 of its top 5 resumes come from exactly the right field, against 0.2 for random ranking. Among the single components, LSA was the strongest (nDCG@10 $= 0.406$), followed by TF-IDF (0.362), tag matching (0.324) and unweighted keyword overlap (0.278). Every method was far above random ranking (0.068). Recall is reported at 100 because a typical gig has about 109 relevant resumes, so even a perfect top-10 list could find less than 10% of them.

Table 2: Test-set ranking quality on real Upwork job posts and real resumes (mean $\pm$ half-width of the 95% confidence interval over 20 random samples; 200 test gigs $\times$ 2,483 resumes per sample). Best value per column in bold.

Method	P@5	R@100	nDCG@10	MRR
Random	0.044 ± 0.003	0.041 ± 0.000	0.068 ± 0.003	0.142 ± 0.008
Keyword	0.236 ± 0.008	0.129 ± 0.002	0.278 ± 0.007	0.438 ± 0.010
Jaccard (tags)	0.277 ± 0.012	0.124 ± 0.003	0.324 ± 0.010	0.462 ± 0.014
TF-IDF	0.314 ± 0.013	0.176 ± 0.004	0.362 ± 0.010	0.484 ± 0.014
LSA	0.348 ± 0.011	0.236 ± 0.006	0.406 ± 0.009	0.541 ± 0.012
Hybrid	**0.389** ± 0.014	**0.237** ± 0.006	**0.438** ± 0.011	**0.566** ± 0.014

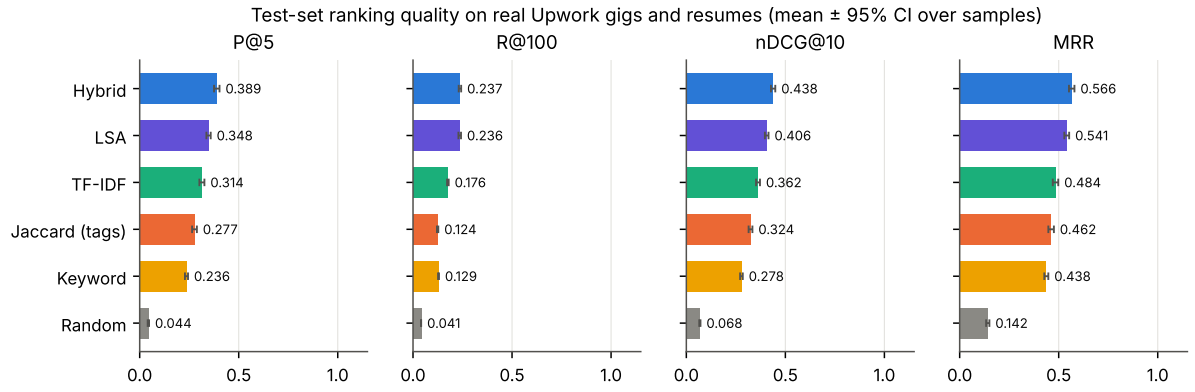


Figure 1: Test-set ranking quality of each method on real data (mean and 95% CI over 20 random samples).

Table 3 shows that the hybrid's advantage is consistent: it beat every baseline on nDCG@10 in all 20 samples, by $+0.114$ over tag matching (paired $t = 29.6$), $+0.076$ over

TF-IDF ($t = 17.5$) and $+0.032$ over LSA ($t = 12.1$); all p-values are below 10^{-4}. On validation data, LSA was chosen as the text component in all 20 samples, with $\alpha = 0.2$ in 17 samples and $\alpha = 0.3$ in 3.

Table 3: Paired comparisons over the same 20 random samples (Hybrid minus baseline).

Baseline	Metric	Mean diff.	Paired t (p)	Wilcoxon p	Hybrid wins
Jaccard (tags)	nDCG@10	+0.114	29.6 (2.3e-17)	1.9e-06	20/20
Jaccard (tags)	P@5	+0.113	21.6 (7.5e-15)	8.8e-05	20/20
TF-IDF	nDCG@10	+0.076	17.5 (3.7e-13)	1.9e-06	20/20
TF-IDF	P@5	+0.075	15.4 (3.5e-12)	8.8e-05	20/20
LSA	nDCG@10	+0.032	12.1 (2.2e-10)	1.9e-06	20/20
LSA	P@5	+0.042	10.9 (1.2e-09)	8.8e-05	20/20

Figure 2 shows the validation curve for sample 0. With LSA text, nDCG@10 rises from 0.342 (tags only, $\alpha = 0$) to a peak of 0.450 at $\alpha = 0.2$ and then slowly falls to 0.415 for text only. With TF-IDF text the best weight is higher ($\alpha = 0.5$, 0.437). In both cases a mixture beats either pure source.

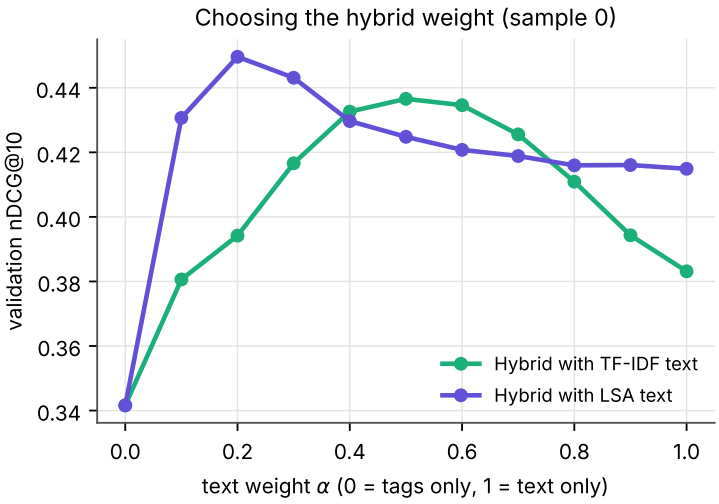


Figure 2: Validation nDCG@10 as a function of the text weight α (sample 0).

Ablation. Table 4 changes one part of the tuned hybrid at a time. Replacing LSA by TF-IDF (re-tuned) lowers nDCG@10 from 0.438 to 0.412; text only gives 0.406, tags only 0.324, and a fixed equal weighting ($\alpha = 0.5$) 0.420. Tuning α on validation data is therefore worth about 0.02 nDCG@10 over a reasonable default.

Differences between kinds of work. Figure 3 breaks the test results down by Upwork category. The hybrid works well for visual and marketing work (*Brand Identity Design* 0.85,

Table 4: Ablation of the hybrid model (test gigs, 20 samples, mean ± 95% CI). α is the weight on text similarity; $1 - \alpha$ goes to tag overlap.

Method	P@5	R@100	nDCG@10	MRR
Hybrid (tuned)	**0.389** ± 0.014	0.237 ± 0.006	**0.438** ± 0.011	**0.566** ± 0.014
Hybrid, other text model	0.363 ± 0.013	0.193 ± 0.004	0.412 ± 0.010	0.537 ± 0.016
text only ($\alpha = 1$)	0.348 ± 0.011	0.236 ± 0.006	0.406 ± 0.009	0.541 ± 0.012
tags only ($\alpha = 0$)	0.277 ± 0.012	0.124 ± 0.003	0.324 ± 0.010	0.462 ± 0.014
equal weights ($\alpha = 0.5$)	0.363 ± 0.012	**0.240** ± 0.006	0.420 ± 0.010	0.551 ± 0.014

SEO 0.76, *Logo Design* 0.76, *Graphic Design* 0.75), and poorly for some technical and support work (*Mobile App Development* 0.19, *Data Analytics* 0.11, *Email, Phone & Chat Support* 0.07). It beat TF-IDF in most of the categories shown. Results per category are in `results/per_category_ndcg.csv`.

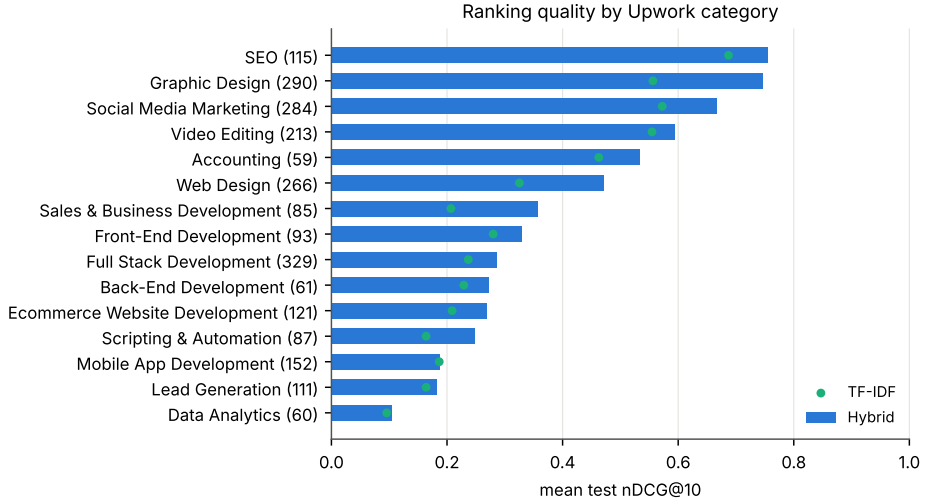


Figure 3: Mean test nDCG@10 for the 15 most frequent Upwork categories (number of test gigs over all samples in brackets).

4.2. Robustness to shorter profiles and missing tags (RQ2)

Real seekers write profiles of very different length and list different numbers of skills. Figure 4 changes one of these at a time (5 samples per level, all 400 gigs, hybrid fixed at LSA with $\alpha = 0.2$). When each resume is cut to its first 100 words, the hybrid keeps almost all of its accuracy (0.440 versus 0.452 for full resumes), while TF-IDF drops from 0.374 to 0.314; with only 25 words the hybrid still reaches 0.366, against 0.288 for LSA and 0.245 for TF-IDF. (Resume tags were taken from the full resume in this test, so tag matching does not change.) When seeker tags are deleted at random, tag matching falls from 0.323 to 0.224 at a deletion probability of 0.8. The hybrid degrades more slowly (0.452 to 0.375) but, with its weight fixed, it drops below LSA alone (0.420) once more than about half of the tags are missing.

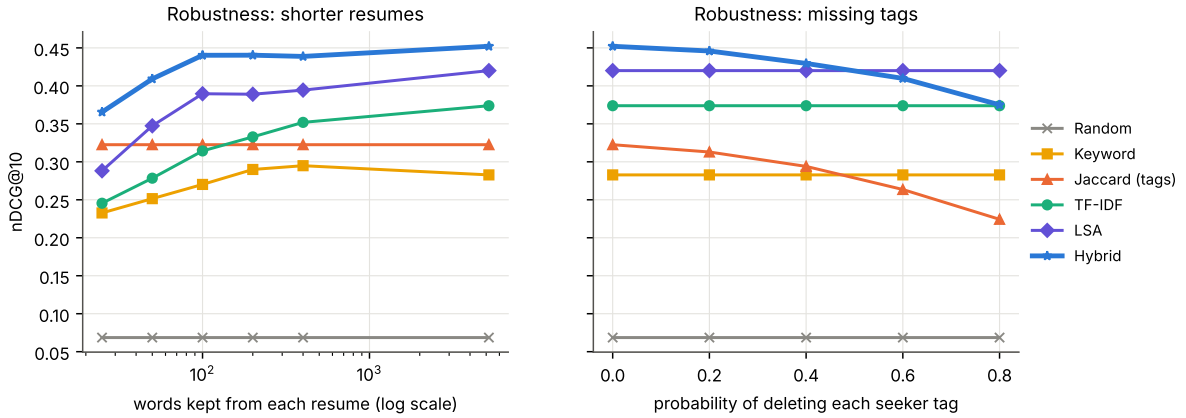


Figure 4: nDCG@10 when resumes are shortened (left) and when seeker tags are deleted at random (right); mean over 5 samples per level.

4.3. Simultaneous assignment (RQ3)

Table 5 matches all 400 sampled gigs at once using the tuned hybrid scores, both with all 2,483 resumes (surplus) and with a random 300 of them (scarce). Exact assignment never double-books and collects more relevance than greedy assignment: 325.8 versus 322.9 (+0.9%) in the surplus market and 129.7 versus 122.2 (+6.1%) in the scarce market. Letting each gig take its own best seeker ("independent top-1") looks best on paper, but it gives the same resumes to many gigs at once (245.7 double bookings with all resumes and 309.6 with 300), which is not a valid plan. The exact solver was also much faster than our greedy loop (0.03 s versus 1.4 s for 400 gigs and 2,483 resumes).

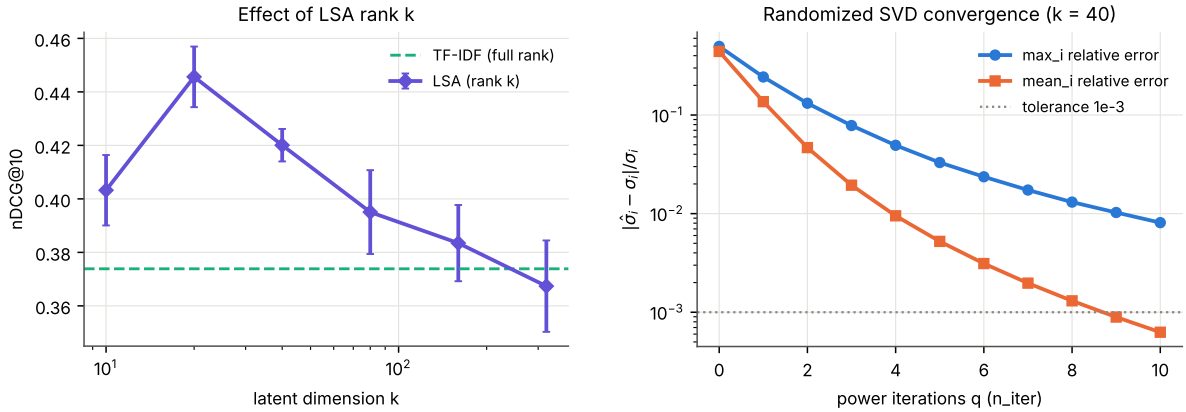
Table 5: Simultaneous assignment of all 400 sampled gigs (mean over 20 samples). *True relevance* sums the category-based relevance of the chosen pairs; *good match rate* is the share of the gigs that receive a resume from the primary matching category ($\mathrm{rel} = 2$).

Market	Policy	Pred. utility	True relevance	Good match rate	Gigs filled	Double-booked
surplus (2483 seekers)	Optimal (Hungarian)	60.13	325.80	0.304	400.0	0.0
	Greedy	58.58	322.85	0.303	400.0	0.0
	Independent top-1	68.85	414.45	0.437	400.0	245.7
scarce (300 seekers)	Optimal (Hungarian)	33.60	129.65	0.115	300.0	0.0
	Greedy	31.96	122.15	0.107	300.0	0.0
	Independent top-1	55.25	364.40	0.367	400.0	309.6

4.4. Numerical behaviour and cost (RQ4)

LSA rank. Figure 5a shows that LSA works best with a small number of concepts: nDCG@10 is 0.403 at $k = 10$, peaks at 0.446 at $k = 20$, is 0.420 at $k = 40$ (our setting, chosen before the sweep), and falls to 0.367 at $k = 320$, slightly below full-rank TF-IDF (0.374). A small k keeps the broad "field of work" of a document and drops the many specific words in which job posts and resumes differ.

Randomized SVD convergence. Table 6 and Figure 5b compare the singular values from the randomized solver with exact ARPACK values for the $2883 \times 41{,}587$ TF-IDF matrix of sample 0 ($k = 40$). The error decreases steadily with the number of power iterations q: with $q = 0$ the mean relative error is 44%; with the setting used in this study, $q = 7$, it is 2.0×10^{-3} (largest 1.7×10^{-2}), and a mean error below 10^{-3} is first reached at $q = 9$. The 40 components explain only 12.5% of the variance, and the exact rank-40 approximation has relative Frobenius error 0.906: LSA keeps a very small, coarse part of the word-level information, which is still enough to identify the field of work.



(a) Effect of the LSA rank k (5 samples, all gigs).

(b) Relative singular-value error versus q.

Figure 5: Numerical behaviour of latent semantic analysis.

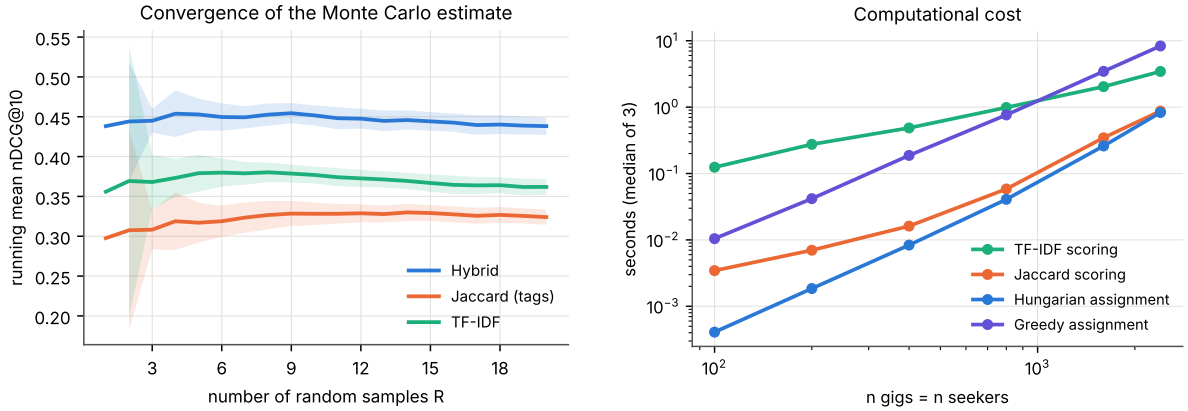
Table 6: Randomized SVD versus exact ARPACK singular values ($k = 40$, matrix 2883×41587, mean of 5 random starts).

q	max rel. error	mean rel. error	explained variance
0	4.97e-01	4.42e-01	0.0437
1	2.43e-01	1.37e-01	0.1060
2	1.32e-01	4.67e-02	0.1191
4	4.93e-02	9.51e-03	0.1239
6	2.36e-02	3.12e-03	0.1246
7	1.74e-02	1.97e-03	0.1248
8	1.31e-02	1.31e-03	0.1249
10	8.10e-03	6.27e-04	0.1249

Monte Carlo convergence. Figure 6a plots the running mean of test nDCG@10 as samples are added. The estimates stabilise after about 10 samples; at 20 samples the 95% confidence half-width is 0.011 for the hybrid model, smaller than the gaps between methods.

Runtime. Table 7 and Figure 6b report wall-clock times for n gigs and n resumes. For a campus-scale case ($n = 400$), TF-IDF scoring takes about 0.5 s and exact assignment under

0.01 s. The fitted log–log slopes for $n \geq 400$ are about 1.1 for TF-IDF scoring (sparse operations), 2.3 for the dense Jaccard matrix, 2.6 for exact assignment (below its $O(n^3)$ worst case) and 2.1 for the greedy method. Our greedy loop is plain Python while SciPy's solver is compiled code, which is why the exact solver was faster in practice. The complete set of experiments took about 17 minutes.



(a) Running mean and 95% CI over samples.

(b) Runtime versus problem size (log–log).

Figure 6: Statistical convergence of the estimates and computational cost.

Table 7: Wall-clock time in seconds (median of 3 runs) for n gigs and n seekers. The last row is the slope of a least-squares line fitted to $\log t$ versus $\log n$ for $n \geq 400$ (empirical complexity exponent).

n	TF-IDF scoring	Jaccard scoring	Hungarian	Greedy
100	0.1244	0.0034	0.0004	0.0104
200	0.2753	0.0070	0.0018	0.0418
400	0.4859	0.0161	0.0084	0.1873
800	0.9879	0.0584	0.0408	0.7631
1600	2.0359	0.3454	0.2600	3.4574
2400	3.4450	0.8729	0.8297	8.3417
slope	1.08	2.26	2.57	2.12

5. Analysis and Discussion

Accuracy. On real data the task is much harder than on a clean, generated dataset: job posts are short and specific, resumes are long and cover a whole career, and the two are written by different people for different purposes. Even so, every content-based method was four to six times better than random ranking, and the hybrid placed a right-field resume first for most gigs (MRR 0.566). An absolute nDCG@10 of 0.44 should be read together with the strict ground truth: a resume counts as fully relevant only when its LiveCareer category is the primary match, so a web designer filed under ARTS gets only partial credit for a *Web Design* gig.

Why the hybrid helps. Tags and text fail in different ways. The Upwork tags of a gig are precise, but a resume's tags come from exact phrase matches and include many generic words (*business*, *development*), so tag overlap alone is noisy. Free text carries the field of work but also much unrelated detail. Combining them (Eq. 3.1) was more accurate than either alone in every sample. The selected $\alpha = 0.2$ puts a weight of 0.8 on tags in the formula, but this does not mean that tags dominate: Jaccard values between a short tag list and a long resume are small (99% of pairs below 0.07, standard deviation 0.013), while LSA cosines spread much more (standard deviation 0.12). After weighting, the text term still varies about twice as much as the tag term, and the tags mainly lift resumes that share the gig's exact skills.

The role of LSA. On real data LSA clearly beat TF-IDF (0.406 versus 0.362), and it did best with only about 20–40 concepts. Real job posts and resumes rarely use the same exact words for the same work, and a low-rank concept space groups words that tend to occur together in one field (for example *photoshop*, *illustrator*, *branding*). This is the vocabulary-mismatch benefit that motivated LSA (Section 2).

Where it fails. The weakest categories are those whose words are shared by many fields or whose workers are spread over several resume categories. *Data Analytics* posts resemble resumes in finance, IT, engineering and consulting; *Email, Phone & Chat Support* posts resemble any customer-facing resume; and software posts are split between INFORMATION-TECHNOLOGY and ENGINEERING resumes. Part of this error comes from the coarse 24-category ground truth rather than from the model.

Stability. Results varied little across samples (standard deviation of the hybrid's nDCG@10 is 0.023), the chosen weights were very stable (LSA with $\alpha \in \{0.2, 0.3\}$ in all samples), and the validation curve is flat near its optimum (Figure 2). Numerically, the randomized SVD converges reliably; at $q = 7$ the singular values are accurate to about two to three significant figures, which does not change the rankings in a meaningful way, and $q = 9$ would meet a 10^{-3} tolerance.

Computational cost. At GigLink's expected scale (hundreds of gigs and seekers) all steps take about a second or less on a laptop, so matching can be recomputed whenever a gig is posted. Exact assignment grows roughly like $n^{2.6}$ in our tests; for thousands of simultaneous gigs the problem could be split by category or restricted to each gig's top candidates.

Practical significance for GigLink. The method can power two features: (i) a ranked "suggested seekers" list on each gig page, with an explanation of which tags and words matched; and (ii) a batch matching step that spreads opportunities across seekers instead of sending every client to the same few people. The per-category results suggest that GigLink can rely on the

matcher first for design, media and marketing gigs, which are also common student gigs, and should treat suggestions for technical and support gigs with more care.

5.1. Limitations and sources of error

- **Proxy data.** The gigs are Upwork posts and the seekers are LiveCareer resumes, not GigLink users. Student profiles will be shorter, and real GigLink posts may be in Filipino or Taglish; the absolute values may differ on GigLink data.
- **Ground-truth definition.** Relevance is judged by job category through a table that we wrote by hand (`data/category_map.csv`). It is coarse (24 resume categories), it reflects our judgement, and it ignores quality, price and availability (Assumption 2). A different table would give different numbers. Real hiring outcomes would be a better ground truth.
- **Constraints not tested.** The resumes contain no schedule, hourly rate or rating, so the feasibility filter (Eq. 3.2) and a reputation term could not be evaluated.
- **Generated tags for seekers.** Resume tags come from exact phrase matching against Upwork tags, which misses synonyms and adds generic words.
- **Static, one-to-one matching.** Real seekers can take several small gigs, and gigs arrive over time. Capacities and online matching are not modelled.
- **Fairness.** Ranking can concentrate opportunities on a few profiles. We did not audit fairness.
- **Hyperparameter selection** used a single random validation split per sample, and the LSA rank $k = 40$ was fixed before the rank sweep, which later showed $k = 20$ to be slightly better.

Table 8: Status of each component of this study.

Component	Status
Real-data loader, category table, TF-IDF/LSA/Jaccard/hybrid models, metrics, assignment	Implemented and unit-tested (9 tests)
Offline evaluation on real Upwork posts and resumes, robustness, convergence and runtime experiments	Run; results reported here
Command-line demo and Google Colab notebook	Implemented; use the real resumes
Schedule and budget constraints	Implemented in code; **not evaluated** (no data)
Integration into the GigLink web application (API endpoint, UI)	**Proposed**, not implemented
Evaluation on GigLink postings and hiring outcomes	**Requires data**; not done
User study, fairness audit, online A/B test	**Proposed**; not done

6. Conclusion

We formulated GigLink's provider-matching module as content-based ranking followed by a linear assignment problem, and implemented it with TF-IDF, latent semantic analysis computed by randomized SVD, Jaccard tag similarity and the Hungarian-type exact assignment solver. On real public data, 42,029 Upwork job posts matched against 2,483 real resumes, the hybrid model reached a test nDCG@10 of 0.438 and a P@5 of 0.389, consistently above LSA (0.406), TF-IDF (0.362), tag matching (0.324) and random ranking (0.068) in all 20 samples. LSA was the most useful text representation on real text, the hybrid stayed accurate even when resumes were cut to 100 words, and accuracy was high for design and marketing gigs but low for data-analytics and support gigs. Exact assignment produced conflict-free simultaneous matches with more total relevance than a greedy heuristic, especially when seekers were scarce, and the whole pipeline runs in about a second at campus scale.

These results suggest that a simple, explainable linear-algebra model is a feasible first AI component for GigLink, and that it can be demonstrated today with the accompanying command-line program and Colab notebook.

7. Recommendations and Future Directions

1. **Validate on GigLink data.** Collect anonymised, consented GigLink postings, profiles and hiring outcomes, re-tune α and k on them, and replace the category table with real outcomes as ground truth.

2. **Evaluate the constraints.** Once seekers state their availability and rates, measure the effect of the schedule and budget filter (Eq. 3.2).

3. **Filipino and Taglish text.** Complement TF-IDF/LSA with multilingual sentence embeddings and compare them under the same evaluation protocol.

4. **Better seeker tags.** Let seekers choose tags from the same list as clients, or learn synonyms, instead of exact phrase matching.

5. **Richer assignment.** Allow seekers to take several small gigs (capacity constraints) and match gigs as they arrive over time (online assignment).

6. **Fairness and integration.** Audit how often new seekers are recommended, and expose the matcher as an API endpoint in the GigLink web application with the explanation (matched tags and words) next to each suggestion.

References

asaniczka. (2024). *Upwork job postings dataset 2024 (50K records)* [Data set]. Kaggle. https://www.kaggle.com/datasets/asaniczka/upwork-job-postings-dataset-2024-50k-records

Bhawal, S. (2021). *Resume dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset

Crouse, D. F. (2016). On implementing 2D rectangular assignment algorithms. *IEEE Transactions on Aerospace and Electronic Systems*, *52*(4), 1679–1696. https://doi.org/10.1109/TAES.2016.140952

de Ruijt, C., & Bhulai, S. (2021). *Job recommender systems: A review* (arXiv:2111.13576). arXiv. https://arxiv.org/abs/2111.13576

Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, *41*(6), 391–407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9

Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, *1*(3), 211–218. https://doi.org/10.1007/BF02288367

Freire, M. N., & de Castro, L. N. (2021). e-Recruitment recommender systems: A systematic review. *Knowledge and Information Systems*, *63*(1), 1–20. https://doi.org/10.1007/s10115-020-01522-8

Halko, N., Martinsson, P.-G., & Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, *53*(2), 217–288. https://doi.org/10.1137/090771806

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, *20*(4), 422–446. https://doi.org/10.1145/582415.582418

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1–2), 83–97. https://doi.org/10.1002/nav.3800020109

Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 73–105). Springer. https://doi.org/10.1007/978-0-387-85820-3_3

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Ren, J., Raghupathi, V., & Raghupathi, W. (2023). Exploring influential factors in hiring freelancers in online labor platforms: An empirical study. *Economies*, *11*(3), Article 80. https://doi.org/10.3390/economies11030080

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, *24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Salton, G., Wong, A., & Yang, C.-S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, *18*(11), 613–620. https://doi.org/10.1145/361219.361220

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, *1*(6), 80–83. https://doi.org/10.2307/3001968

Zhao, J., Wang, J., Sigdel, M., Zhang, B., Hoang, P., Liu, M., & Korayem, M. (2021). *Embedding-based recommender system for job to candidate matching on scale* (arXiv:2107.00221). arXiv. https://arxiv.org/abs/2107.00221

A. Reproducing the Results

All code, results and this paper's source are in the project's GitHub repository. The two raw Kaggle files are not stored in the repository; download them (asaniczka, 2024; Bhawal, 2021) into `data/raw/` first (see `data/raw/README.md`). From the repository root:

```
python -m venv .venv && source .venv/bin/activate # Windows: .venv\
    Scripts\activate
pip install -r requirements.txt
python -m pytest -q # 9 unit tests
python -m giglink_ai.experiments # all experiments, figures and tables
python -m giglink_ai.demo # try the matcher on a sample gig
cd paper && latexmk -pdf main.tex # rebuild this paper
```

Because every random draw is seeded, the metric values are identical on every run; only the runtime measurements depend on the machine. The notebook `notebooks/GigLink_Colab.ipynb` runs the same code in Google Colab.

B. Ranking Metrics Code

Listing 5: Precision@k and nDCG@k (excerpt of `giglink_ai/metrics.py`).

```
def _topk(scores, k):
    order = np.argsort(-scores, axis=1, kind="stable")
    return order[:, :k]

def precision_at_k(scores, rel, k=5, threshold=2):
    top = _topk(scores, k)
    hits = np.take_along_axis(rel, top, axis=1) >= threshold
    return float(hits.mean())

def ndcg_at_k(scores, rel, k=10):
    discounts = 1.0 / np.log2(np.arange(2, k + 2))
    top = _topk(scores, k)
    gains = 2.0 ** np.take_along_axis(rel, top, axis=1) - 1
    dcg = (gains * discounts).sum(axis=1)
    ideal = 2.0 ** (-np.sort(-rel, axis=1)[:, :k]) - 1
    idcg = (ideal * discounts).sum(axis=1)
    mask = idcg > 0
    return float((dcg[mask] / idcg[mask]).mean())
```